PW EARNERS

DSA in C++

Daily Practice Problems (DPP) — 10

Topic: Recursion

Multiple Choice Questions (Theory & Predict the Output)

For Q1 to Q8: Select the correct option. Assume all required headers like `<iostream>` *are included, and* `using namespace std;` *is present.*

Q1. What will be the output of the following recursive function?

```
int f(int n) {
    if (n <= 1) return 1;
    return f(n - 1) + n;
}
int main() {
    cout << f(5);
}
```

A. `10` B. `15` C. `120` D. `Compilation Error`

Q2. What is the overall time complexity of the following recursive function?

```
void solve(int n) {
    if (n <= 1) return;
    for (int i = 0; i < n; i++) {
        cout << "*";
    }
    solve(n / 2);
    solve(n / 2);
}
```

A. $O(N)$ B. $O(N^2)$ C. $O(N \log N)$ D. $O(2^N)$

Q3. Predict the output of the following code snippet:

```
void modify(int &x, int c) {
    if (c == 0) return;
    x = x + 2;
    modify(x, c - 1);
}
int main() {
    int p = 5;
    modify(p, 3);
    cout << p;
}
```

A. 5 B. 11 C. 8 D. 13

Q4. What will be the output of the following C++ code snippet?

```
void funB(int n);
void funA(int n) {
    if (n <= 0) return;
    cout << n << " ";
    funB(n - 1);
    cout << n << " ";
}
void funB(int n) {
    if (n <= 1) return;
    cout << n << " ";
    funA(n / 2);
}
int main() {
    funA(5);
}
```

A. 5 4 2 2 5

B. 5 4 2 1 2 4 5

C. 5 4 2 1 1 2

D. Infinite Recursion

Q5. Predict the output of the following tree recursion code:

```
void printNum(int n) {
    if (n == 0) return;
    cout << n << " ";
    printNum(n - 1);
    cout << n << " ";
}
int main() {
    printNum(3);
}
```

A. 3 2 1 1 2 3

B. 1 2 3 3 2 1

C. 3 2 1 0 1 2 3

D. 3 3 2 2 1 1

Q6. *(Crazy Question)* Predict the output of the following nested recursive function:

```
int fun(int n) {
    if (n > 100)
        return n - 10;
    return fun(fun(n + 11));
}
int main() {
    cout << fun(95);
}
```

A. 95 B. 101 C. 91 D. 85

Q7. *(Edge Case Challenge)* What happens when the following code is executed?

```cpp
void crashCourse(int n) {
    if (n == 0) return;
    cout << n << " ";
    crashCourse(n--);
}
int main() {
    crashCourse(3);
}
```

A. Prints `3 2 1` and terminates gracefully

B. Prints `2 1 0` and terminates gracefully

C. Results in a Stack Overflow (Infinite Recursion)

D. Compilation Error due to modifying parameter

Q8. Fill in the blanks (marked as `___1___`, etc.) to complete the recursive implementation of the **Binary Search** algorithm:

```cpp
int binarySearch(int arr[], int low, int high, int target) {
    // Base Case
    if (___1___) return -1;

    int mid = ___2___;

    if (arr[mid] == target) return ___3___;

    if (arr[mid] > target)
        return ___4___;

    return ___5___;
}
```

A. **1:** `low > high`,
2: `low + (high - low)/2`,
3: `mid`,
4: `binarySearch(arr, low, mid - 1, target)`,
5: `binarySearch(arr, mid + 1, high, target)`

B. **1:** `low >= high`,
2: `(low + high)/2`,
3: `arr[mid]`,
4: `binarySearch(arr, low, mid, target)`,
5: `binarySearch(arr, mid, high, target)`

C. **1:** `low == high`,
2: `low + high / 2`,
3: `target`,
4: `binarySearch(arr, low - 1, mid, target)`,
5: `binarySearch(arr, mid, high + 1, target)`

D. **1:** `low < high`,
2: `mid / 2`,
3: `-1`,
4: `binarySearch(arr, mid - 1, high, target)`,
5: `binarySearch(arr, low, mid + 1, target)`

*For Q9 to Q15: Write clean, optimal, and compilable C++ programs strictly using **Recursion** (no loops unless absolutely necessary for setting up the initial call). The questions are arranged in increasing order of difficulty.*

Q9. Reverse a String. Write a recursive function that takes a string (passed by reference) and reverses it in-place. Do not use any built-in functions like `reverse()` or loop structures.
Constraints:
$1 \leq$ s.length $\leq 10^5$

Sample Input: `s = "hello"`
Sample Output: `"olleh"`

Q10. Valid Palindrome. Write a recursive algorithm to check if a given string is a palindrome. The function should return a boolean value. A string is a palindrome if it reads the same forwards and backwards.
Constraints:
$1 \leq$ s.length $\leq 10^5$

Sample Input 1: `s = "racecar"`
Sample Output 1: `true`
Sample Input 2: `s = "apple"`
Sample Output 2: `false`

Q11. Power of Three. Given an integer `n`, return `true` if it is a power of three. Otherwise, return `false`. Solve this strictly using recursion without using any loops or the built-in `pow()` function.
Constraints:
$-2^{31} \leq n \leq 2^{31} - 1$

Sample Input 1: `n = 27`
Sample Output 1: `true`
Sample Input 2: `n = 0`
Sample Output 2: `false`

Q12. GCD of Two Numbers (Multiple Queries). You are given Q queries. In each query, you are given two integers A and B. Find the Greatest Common Divisor (GCD) of A and B using recursion. To pass all test cases, your recursive algorithm must run in strict $O(\log(\min(A, B)))$ time complexity per query.
Constraints:
$1 \leq Q \leq 10^5$
$1 \leq A, B \leq 10^9$

Sample Input:
```
3
24 60
9 15
17 13
```
Sample Output:
```
12
3
1
```

Q13. Check if Array is Sorted. Given an array of integers of size N, write a recursive function to check if the array is sorted in strictly increasing order. Return `true` if it is, otherwise return `false`.
Constraints:
$1 \leq N \leq 10^5$

Sample Input 1: `arr = [1, 2, 4, 8, 9]`
Sample Output 1: `true`

Sample Input 2: `arr = [1, 3, 2, 4]`
Sample Output 2: `false`

Q14. Decode String. Given an encoded string, return its decoded string. The encoding rule is: `k[encoded_string]`, where the `encoded_string` inside the square brackets is being repeated exactly `k` times. Note that `k` is guaranteed to be a positive integer. You may assume that the input string is always valid and contains no extra spaces.

Hint: *Think about how to maintain an index reference globally or pass it by reference across your recursive calls to parse nested brackets effectively.*

Constraints:

$1 \leq$ s.length ≤ 30

`s` consists of lowercase English letters, digits, and square brackets `'[]'`.

Sample Input 1: `s = "3[a]2[bc]"`
Sample Output 1: `"aaabcbc"`

Sample Input 2: `s = "2[ab3[cd]]"`
Sample Output 2: `"abcdcdcdabcdcdcd"`

Q15. K-th Symbol in Grammar (Hard). We build a table of n rows (1-indexed). We start by writing `0` in the 1st row. Now in every subsequent row, we look at the previous row and replace each occurrence of `0` with `01`, and each occurrence of `1` with `10`.

Given two integers n and k, return the kth (1-indexed) symbol in the nth row of a table of n rows.

Note: *This problem uses basic knowledge of bitwise operations. If you haven't learned bitwise operators yet, feel free to skip this question for now and revisit it later!*

Constraints:

$1 \leq n \leq 30$

$1 \leq k \leq 2^{n-1}$

Sample Input 1: `n = 4, k = 5`
Sample Output 1: `1`
Explanation 1:
Row 1: `0`
Row 2: `01`
Row 3: `0110`
Row 4: `01101001`
The 5th symbol in Row 4 is `1`.

Sample Input 2: `n = 5, k = 11`
Sample Output 2: `0`
Explanation 2:
Row 5 is `0110100110010110`. The 11th character from the left is `0`.

All the best! Keep Coding ♡